

Scheda — FI2 LAB05: TicketSosta

Materiale: LAB-05-TicketSosta.pdf (sl. 1–35), Lab05-TicketSosta-Startkit.zip (Tariffa, 4 classi di test) · **Modalità:** guidata · **Tempo del docente:** 45 min parte 1 [sl. 14], 60 min parte 2 [sl. 26] **Teoria che entra qui:** 10 □ *Date* — `LocalTime`, `LocalDateTime`, `Duration`, `DayOfWeek` (sl. 13) · 11 □ *Formati* — `NumberFormat`, `Locale.ITALY` (sl. 11) · S02 □ *JUnit* (sl. 29–35). Tre moduli ancora □: entrano qui, 5–10 righe l'uno quando compaiono, si chiudono coi test verdi.

Setup

- *Existing Projects* → *Select root directory* → `~/cockpit/Vault/UniCode/corsi/FI2/esame_FI2/051026` → **Copy projects into workspace**; poi **rinomina il progetto** (all'esame è richiesto).
- Due cartelle sorgenti: `src` (codice) e `tests` (test, già pronti) [sl. 10]. Compliance 21.
- X rosse attese: mancano `Ticket`, `Parcometro` (package `ticketsosta`) e, nella parte 2, il package `ticketsostaevoluto`. Tariffa c'è già e **non si tocca** [sl. 17].
- ⚠ Se `org.junit.Assert` non si risolve (`ParcometroTest` e `ParcometroEvolutoTest` lo importano, JUnit 4): cambia l'import in `org.junit.jupiter.api.Assertions`. Se manca JUnit del tutto: sl. 31–33.

Ordine di lavoro

#	Classe	Cosa rappresenta	Test che la coprono	Costrutto nuovo → slide
1	<code>Ticket</code>	biglietto: inizio, fine, costo; classe-dati	<code>TicketTest</code>	<code>toString</code> personalizzato, <code>getCostoAsString</code> con <code>NumberFormat</code> , spazio non separabile → sl. 11
2	<code>Parcometro</code>	tiene la Tariffa, calcola il costo, emette il <code>Ticket</code>	<code>ParcometroTest</code> (6 test)	<code>Duration.between</code> , <code>isBefore</code> , mezzanotte → sl. 12–13
3	<code>TicketEvoluto</code>	come <code>Ticket</code> con <code>LocalDateTime</code>	<code>TicketEvolutoTest</code>	sl. 18–19

#	Classe	Cosa rappresenta	Test che la coprono	Costrutto nuovo → slide
4	ParcometroEvoluto	tariffa[] (0 = lunedì...), calcolaCostoSuPiuGiorni, toString su tutte le tariffe	ParcometroEvolutoTest (7 test)	DayOfWeek, ciclo sui giorni → sl. 20-23

Algoritmo di calcolo: sl. 8 e 23. Firme: nei test e nell'UML di sl. 22.

Casi limite da pensare

- Fine alle 00:00 (mezzanotte): `a.isBefore(da)` è vero ma la sosta è valida. Che durata esce? (sl. 12)
- Franchigia **più lunga** della sosta: il tempo da pagare diventa negativo? E la durata minima, si applica prima o dopo la franchigia? Verificalo sui test H1f.
- `costo orario × durata`: durata in minuti o in ore? Cosa dà $0.50 \times 450 / 60$? Il tipo `int` ti tradisce? (△ divisione intera, LAB03)
- Evoluto: la franchigia vale **solo sul primo giorno** [sl. 23]. Quadra su `testSostaDueGiorni...` (60,25): quanto paga il primo giorno, quanto il secondo?
- Evoluto: come passi da `DayOfWeek` all'indice 0-6 dell'array? (△ pattern 5: lunedì è 0 o 1? derivalo da un caso, non a memoria)
- Evoluto: la durata minima vale per ogni giorno o per tutta la sosta? Il test `1130_1150` (1,50 €) te lo dice.
- △ Pattern 2: la classe è finita quando passano **tutti** i test, non il primo; prova a mano un caso tuo (sosta di 0 minuti, sosta a cavallo di mezzanotte).

Test rosso?

Non compila (firma/nome/package) · failure (guarda il valore atteso: spazio , virgola decimale) · error (eccezione: stack trace) → prontuario §1, oppure scrivilo in chat.

Dopo i test verdi

- copia i due package in `corsi/FI2/esame_FI2/svolti/LAB05_TicketSosta/`;
- apri `Lab05-TicketSosta-Soluzione.zip` e confronta: algoritmo, casi limite, metodi privati;
- Claude scrive `svolti/LAB05_TicketSosta/confronto_LAB05.md` (modello: `svolti/LAB02_Frazione/confronto_LAB02.md`);
- /chiudi aggiorna il prontuario (§ model: `java.time`, `formattatore valuta`).

Famiglia d'esame ★

Tipologia: caso d'esame di luglio 2018 + refactoring su nuovi requisiti (design for change, sl. 27). Parte 1 = compito intero; parte 2 = cosa cambia quando i requisiti cambiano.